Object Detection Learning Techniques for Autonomous Vehicle Applications

Mehdi Masmoudi[1,2], Hakim Ghazzai[2], Mounir Frikha[1], and Yehia Massoud[2]
[1]University of Carthage, Higher School of Communications of Tunis, Tunisia
Email: {mehdi.masmoudi, m.frikha}@supcom.tn
[2]School of Systems & Enterprises, Stevens Institute of Technology, Hoboken, NJ, USA
Email: {mmasmoud, hghazzai, ymassoud}@stevens.edu

***Abstract*— Autonomous vehicles have been considered as one of the most important trending topics in the domain of intelligent transportation systems. It is expected that most of the leading companies will launch their fully self-driving vehicles by the end of the next decade. Such a technology must assure a high safety level on road networks to reduce the number of accidents caused by human errors. All the built-in technologies must be integrated and complemented to achieve these goals. Automated object and obstacle detection is one of the main research tasks that must be undertaken. A number of vision-based learning techniques have been designed to improve the vehicle detection capabilities and reduce the shortcomings of other sensors such as LIDAR system that have shown poor results during severe weather conditions. In this paper, we overview and investigate the main learning models for video-based object detection specifically that can be applied with autonomous vehicle. We focus on a machine learning solution, namely, the Support Vector Machine (SVM) algorithm and two deep learning solutions: the "You Only Look Once" (YOLO) and the Single Shot Multibox Detector (SSD) methods. Afterwards, we provide an exhaustive comparison of the different approaches when applied to autonomous vehicles. Simulations results indicate that the SVM poorly performs and its speed can not assure real-time response, while the YOLO model and SSD can reach higher accuracy with a notable ability to detect objects in real-time when rapid driving decisions need to be made.**

***Index Terms*—Autonomous vehicle, deep learning, image processing, machine learning, vehicle detection.**

I. Introduction

Over the last few years, autonomous vehicles have attracted a lot of attention in both research and industrial domains and have witnessed a perpetual evolution since then. Many innovative projects are being undertaken to improve and design the self-driving process of these smart vehicles in order to deliver operational and safe automated technology that will change the pattern of transportation.

At present, such vehicles have the capability to drive, slow down, stop, and even park by themselves. Google is one of the pioneer companies to start the prototyping of fully self-driving vehicles. These vehicles are equipped with advanced sensors such as ultrasonic, RADAR, Light Detection and Ranging (LIDAR), as well as a series of cameras with 360 degrees of view. The purpose is to overcome obstacles, avoid collision, and ensure a safe navigation of the autonomous vehicle especially in congested areas. While great progress has been achieved, catastrophic failures have happened and the list of incidents is getting longer. For instance, in 2018, a fatal crash caused the first death of a pedestrian caused by a full autonomous car[1]. Therefore, there is a need to devise additional advanced and complementary solutions to further improve the capabilities of these vehicles. Every contribution obtained from one of the on-board devices can complement the rest of the sensors and help in providing better and safer driving.

Recently, many object detection, recognition, and tracking approaches have been proposed in the literature [1], however, few of them were designed to improve the efficiency of the self-driving vehicles. In [2], the authors developed a subsystem to handle pedestrians in crosswalks by implementing a deep neural network algorithm for the Intelligent Autonomous Robotic Automobile (IARA) by using LIDAR and cameras. Pedestrians can be detected by the camera using a Convolutional Neural Network (CNN) and the vehicle positions can be calculated by fusing the position in the image with the LIDAR point cloud. In [3], the authors proposed a novel approach to detect vehicles and recognize brake lights from rear appearance images in real-time using a deep classifier to classify the vehicle as 'normal' or 'brake'. Road segmentation and vanishing points are used to improve the accuracy and speed up the segmentation process. In [4], an end-to-end pipeline to detect and categorize road-objects like cars, cyclists, and pedestrians is presented. The pipeline, named SqueezeSeg, is based on a CNN refined by a Conditional Random Field (CRF) for real-time road-object segmentation from 3D LIDAR point cloud. The authors of [5] proposed a traffic-sign detection and extraction module. They employed a scoring SVM model. First, traffic sign color and the color gradient are extracted. Then, the shape of traffic sign is computed by a voting scheme yielding shape score maps, which will be used to train the SVM model. In [6], the motion planning problem for autonomous vehicles is exposed. Considered as one of the most important technologies in this field, the main idea of the study is to deal with emergency situations. The authors developed a framework named 'parallel planning' which is a combination of CNN and Long Short-term Memory (LSTM). The proposed model can learn from both artificial and real roads to then, imitate the behaviour of human drivers.

[1]https://www.nbcnews.com/business/autos/fatal-crash-could-pull-plug-autonomous-vehicle-testing-public-roads-n858151

978-1-7281-3473-4/19/$31.00 ©2019 IEEE

As technologies evolve, different shortcomings of the current models used in autonomous vehicles have been noticed. Taking LIDAR as an example, the sensor is too expensive and there are critical perception errors. In fact, the performance of this system starts to fail for severe weather conditions such as rainy and snowy weathers. For that reason, companies like Google are using LIDAR with other complementary sensors either ultrasonic sensors, cameras, and/or radar systems.

In this paper, we overview the main object detection learning techniques using image processing for autonomous vehicle applications. The purpose is to investigate the performance of machine learning solutions as well as deep learning algorithms based on the accuracy and the timing-process. We start by presenting the architecture of the three approaches: namely the machine learning SVM model, the deep learning You Only Look Once (YOLO) approach, and the Single Shot Multibox Detector (SSD) deep learning model. Afterwards, we provide extensive simulations to evaluate the performance, in terms of accuracy and frame processing speed, of these object detection approaches in identifying vehicles on roads. Results show that SVM poorly performs and its speed cannot assure real-time response. With higher frame processing speed, the YOLO model and SSD can meet the requirements of object detection for autonomous vehicle applications.

II. Object Detection with SVM

In this section, we introduce the architecture of the object detector SVM. SVM is a supervised learning model that analyzes data for both regression and classification. The structure of the model consists of four main stages as shown in Fig. 1:

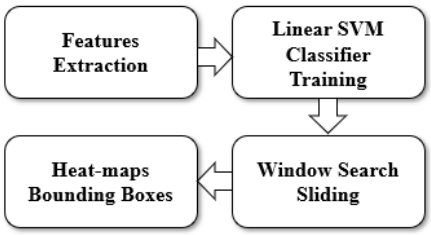

Fig. 1: Workflow for object detection with SVM.

Features Extraction: In this part, some feature engineering are applied on the data set in order to find the ideal features for the model. To this end, three descriptors are used to extract different types of features:

i) Histogram of Oriented Gradients (HOG): HOG is a feature descriptor corresponding to a representation of a simplified image by extracting useful information by separating the background and the target [7]. It converts an image to a feature vector of length $W \times H \times 3$ where W is the width of the image, H is its height, and three is the number of RGB (Red, Blue, Green) color channels.

ii) Spatial Binning: The spatial binning consists of the collection of the image pixels as a feature vector where close pixels are combined together to form a lower resolution image.

iii) Color Distribution: The color distribution determines the histogram for each color channel. It combines the three histograms to produce one feature vector.

A feature vector is obtained from each feature extraction step. The three obtained vectors are concatenated to form a single feature vector to be used as an input for the SVM training stage.

SVM Training: The linear SVM is employed to detect objects since it offers the best trade-off between accuracy and computational speed [8]. In fact, linear SVM is less complex than the kernel SVM and trained on HOG features can achieve good performance by detecting target objects even in difficult situations [9].

Window Search Sliding: Afterwards, a sliding-window search is applied to locate objects on the image. The sliding-window size is chosen by the designer. By sweeping the whole images, several objects or part of objects are detected in different parts of the image. This makes the image split into overlapping areas (windows) to fed the model for prediction.

Heat-maps Bounding Boxes: As some detected objects are false positives, which means false detection of the target object, a threshold is needed to select only the areas with overlap. The result is a heat-map threshold with reduced noise to better identify the detected objects.

III. Object Detection With YOLO

In this section, we introduce the architecture of the object detector YOLO. YOLO is a fully CNN (FCN). It contains only convolutional layers and is invariant to the size of the input image. YOLO takes a different approach to object detection since it "looks" at each image once as indicates its name: You Only Look Once [10]. YOLO converts the input image into a grid by $S \times S$ cells and each of these cells is subject for the prediction of N bounding boxes enclosing the object. For each bounding box, a particular object belonging to a specific class is detected with a score representing the level of confidence. Classes can be cars, buses, pedestrians, trucks, traffic lights, and all other things showing up along the way. The structure of the model consists of four main stages as shown in Fig. 2.

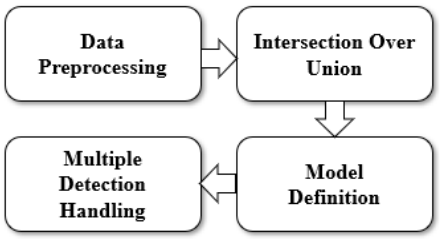

Fig. 2: Workflow for object detection with YOLO.

Data Preprocessing: Before getting into the object detection model, we need to pre-process the data so as to resize the image and convert its bounding box coordinates into useful information. We define the output variable as:

$$\hat{y} = [p_c, b_x, b_y, b_h, b_w, c_1, c_2, \ldots, c_m]^\intercal, \quad (1)$$

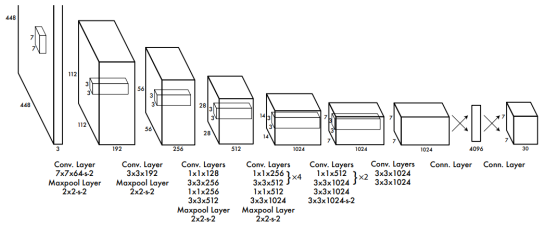

Fig. 3: The YOLO Architecture [10].

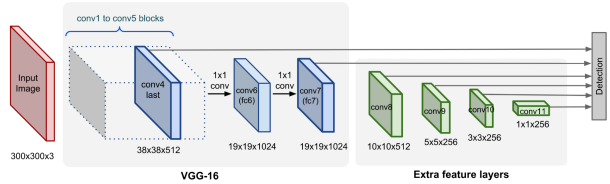

Fig. 4: The SSD architecture [11].

where p_c is the confidence of a detected object while b_x and b_y are the bounding box center coordinates expressed as follows:

$$b_x = \frac{(x_{min} + x_{max})}{2\,W} \text{ and } b_y = \frac{(y_{min} + y_{max})}{2\,H}. \quad (2)$$

In (1), b_w and b_h correspond to the bounding box width and height, respectively, and are given as follows:

$$b_w = \frac{(x_{min} - x_{max})}{2\,W} \text{ and } b_h = \frac{(y_{min} - y_{max})}{2\,H}. \quad (3)$$

Finally, c_i denotes the ith class probability.

Intersection Over Union: To measure the accuracy of the algorithm and evaluate its performance, an evaluation metric named the Intersection Over Union (IOU) is defined as a metric representing the overlap between boxes detected. The IOU is the ratio of the area of overlap and the area of union. This score shows how much the predicted bounding box matches the ground-truth bounding box in the image.

Model Definition: To create a final model, a transfer learning is applied from a pre-trained network. YOLO involves one neural network to the image, has 24 convolutional layers, and two fully connected layers as shown in Fig. 3. The convolutional layers are pre-trained on the ImageNet classifier.

Multiple Detection Handling: To detect and localize multiple objects in the test image, we need for thresholds and suppression to fix these detections.

i) Threshold Filtering: In order to filter out all bounding boxes that do not contain the target class, a filter discarding the objects having low confidence probability. Hence, if p_c is less than the threshold chosen by the object detection designer (e.g., 20% or 30%), the corresponding box is eliminated from the list of boxes.

ii) Non-max Suppression: Even if the boxes are already filtered out, it is still possible that some of them are overlapping due to duplicate detection. To this end, a second filter, named Non-Maximal Suppression (NMS), is used to eliminate boxes duplicately detected.

iii) Anchor boxes: With the idea of anchor boxes, multiple objects or classes can be simultaneously detected in an image. Hence, if we use N anchor boxes, the system can detect at maximum N objects in the input image.

IV. Object Detection With SSD

The SSD is one of the first methods using a single convolutional network's pyramidal feature hierarchy for different size object detection [11]. The pyramidal hierarchy consists of several convolutional layers of decreasing sizes. Unlike YOLO, SSD does not split the input image into grids. It already includes multi-scale detection and default boxes. The multi-scale detection increases the robustness of the detection by choosing windows of different sizes to detect objects independently. SSD uses lower resolution layers to push the computational speed higher and detect large scale objects.

Model Approach: The approach of this algorithm is based on a feed-forward CNN followed by a NMS filter. SSD contains two structures: a base and an auxiliary structure. The base structure is used for classification and the auxiliary one is used for detection with the following features:

i) Multi-scale Detection: Convolutional feature layers are added to the end of the base structure with decreasing size to allow the prediction of bounding boxes at multiple scales. These layers produce feature maps with different sizes: 19×19, 10×10, 5×5, 3×3, and 1×1. An example is provided in Fig. 4. The 38×38 grid is used to detect smallest objects while the 1×1 grid is dedicated for detecting objects taking up the whole image.

ii) Default Boxes and Aspect Ratios: Default boxes are similar to anchors in YOLO. These boxes are pre-selected to cover the spectrum of real-life objects with fixed-size collection and different scales or aspect ratios in a single layer.

Training Method: The training includes determining default boxes and scales, hard negative mining, and data augmentation actions:

i) Matching Action: It matches default boxes to a ground truth box with an overlap higher than a threshold chosen by the designer.

ii) Aspect Ratios and Scales Selection: The scale of each default box for each feature map is calculated as follows:

$$s_k = s_{\min} + \frac{(s_{\max} - s_{\min})}{m-1}\,(k-1). \quad (4)$$

where $s_{\min} = 0.2$ is the scale of the lowest layer, $s_{\max} = 0.9$ is the scale of the highest layer, and m is the number of features maps. The aspects ratios can have the following values $\{1, 2, 3, \frac{1}{2}, \frac{1}{3}\}$.

iii) Hard Negative Mining: To achieve stable training and faster optimization, the negative examples (false positive) are sorted in the ascending order and most of the wrong examples are picked up in the loss function.

iv) Data Augmentation: A zoom in and zoom out actions are used to increase the performance of detecting large and small objects, respectively, before feeding the network.

V. Experimental Results and Performance Comparisons

In our simulations, we choose a video of 50 seconds taken from the front camera of the Udacity self-driving car. The 50

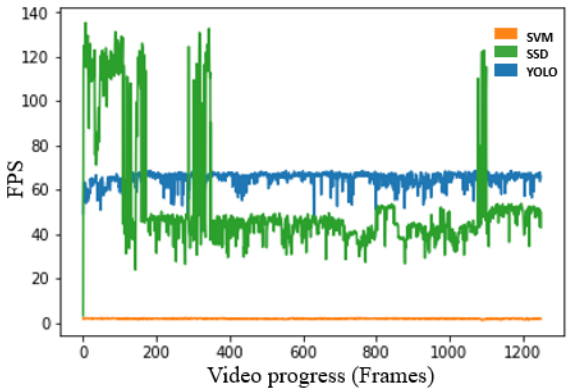

Fig. 5: FPS versus the video progress for SVM, YOLO, and SSD.

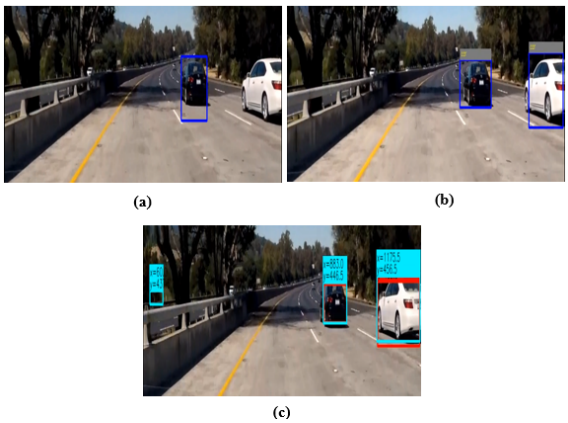

Fig. 6: Snapshots taken at the same instant showing the detected vehicles using (a) SVM, (b) YOLO, and (c) SSD.

seconds video is sub-divided into 1250 frames. In other words, an image is captured every 40 ms. Then, the set of frames is fed to the learning models to detect vehicles.

We start by investigating the number of frames treated by each model: the Frame Per Second (FPS) in Fig. 5, which reflects the speed of each learning model in treating the frames. We can notice that SVM poorly performs compared to YOLO and SSD. SVM is very slow and its speed cannot exceed 2 FPS because of its incorporated sliding window approach. Thus, this model cannot run for real-time applications. Regarding the remaining approaches, YOLO has shown a very fast processing time and can achieve 50-70 FPS due to the fact that it looks at the image once. On the other hand, the SSD can attain on average 40-60 FPS and does work real-time object detection where rapid driving decisions should be made. Due to the absence of objects in some frames, SSD presents spikes where it can generate the output result rapidly.

In Fig. 6, we provide three instantaneous snapshots of the recorded video showing the detection of the three algorithms. The SVM, shown in Fig. 6(a), cannot detect all presented cars. Its boxes are smaller than the size of the detected object and are unstable which affects its accuracy. However, the YOLO model, illustrated in Fig. 6(b), detects better the nearby vehicles but struggles with smaller objects unlike the SSD, which can reach higher accuracy due to the presence of default boxes and multi-scale features as shown in Fig. 6(c). The SSD has more portability to detect vehicles even moving in the opposite direction.

In Fig. 7, we describe the number of detected objects per frame for the input video where true and false objects (i.e., vehicles and non-vehicles) are taken into account. The SSD, shown in Fig. 7(c) makes more predictions than the YOLO, illustrated in Fig. 7(b), due to the presence of grids and detectors in the higher resolution layer so it can detect small objects. The SVM displays multiple boxes for the same object detected in view of the absence of NMS, as shown in Fig. 7(a), and make false detection where it detects objects other than vehicles which affects its accuracy.

In Fig. 8, we evaluate the performance of the learning models in performing true detection. In other words, we compare the number of detected vehicles and the number

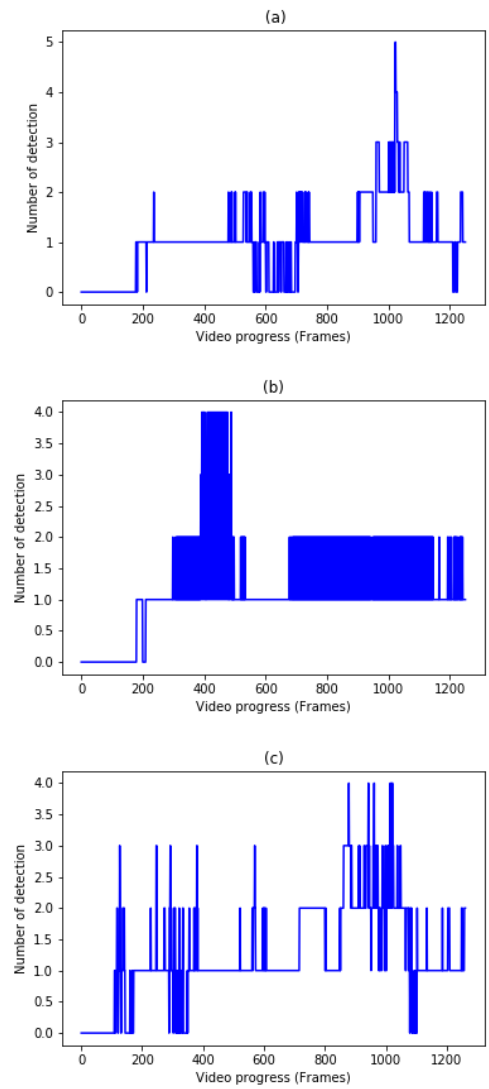

Fig. 7: Number of detected objects versus the video progress with (a) SVM, (b) YOLO, and (c) SSD.

of existing vehicles during the duration of the input video. The existing vehicles include all the vehicles moving in both directions. The SVM, shown in Fig. 8(a) cannot detect all

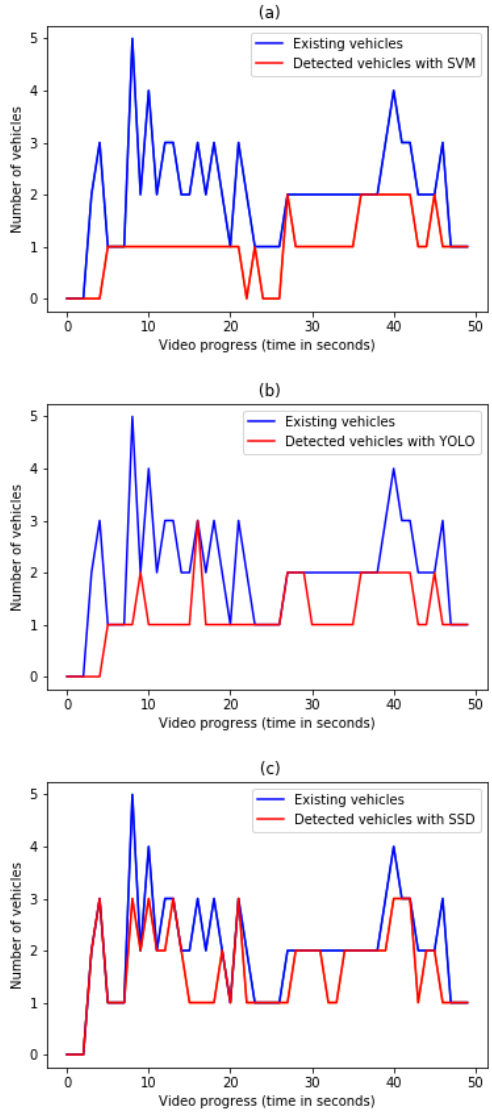

Fig. 8: Number of detected and actual existing vehicles versus time with (a) SVM, (b) YOLO, and (c) SSD.

the presented vehicles and struggles with cars going in the opposite direction. However, the YOLO model, illustrated in Fig. 8(b) detects more vehicles, but still poorly performs where objects are faraway from the camera, unlike the SSD, where vehicles going in the opposite direction and small objects have higher chance to be detected as shown in Fig. 8(c).

TABLE I: Efficiency comparison.

Models	True detection	False detection	Non detection
SVM	50%	18%	50%
YOLO	59%	9.23%	41%
SSD	81%	4.93%	19%

In Table I, we provide the efficiency of each model where we evaluate the percentage of true detection (i.e., percentage of detected vehicles), false detection (i.e., percentage of non-vehicle detected objects and double detection), and non-detection (i.e., percentage of vehicles that are not detected) in the input video. SVM can only detect 50% of them and make false predictions with a rate of 18%. The YOLO can detect more vehicles than the SVM with a rate of 59% but make false predictions with a rate of 9.23%. Note that YOLO does not fail in detecting wrong objects but it only performs some double detection. The SSD is the most accurate model and can predict 81% of the present vehicles with a very low false detection rate (only 4.93%).

VI. Conclusion

In this paper, we have investigated the performance of different learning models in detecting objects for real-time autonomous vehicle applications. The empirical results show the existence of a trade-off between high performance and real-time prediction capability. SVM is very slow and cannot run for real-time applications. Therefore, machine learning solutions, in general, are not suitable for real-time object detection. Deep learning solutions such as YOLO and SSD provide effective vehicle detection results and can be implemented to complement other technologies in detection vehicles and obstacles. YOLO is very fast due to the fact that it looks at the image once but provides worse performance than SSD, which employs multi-scale features. Nevertheless, the SSD still suffers from a lower speed in treating frames. Depending on the applications' objectives, YOLO should be employed for extremely real-time processing due to its rapidity in treating the frame while SSD can be employed for its high accuracy in detecting small objects.

References

[1] G. Sapijaszko and W. B. Mikhael, "An overview of recent convolotional neural networks algorithms for image recognition," in *IEEE Int. Midwest Symposium Cir. Syst. (MWSCAS'18)*, Windsor, ON, Canada, Aug. 2018.

[2] R. Guidolini et al., "Handling pedestrians in crosswalks using deep neural networks in the IARA autonomous car," in *IEEE Int. Joint Conf. Neural NetW (IJCNN'18), Rio, Brazil*, Jul. 2018.

[3] J. Wang, L. Zhou, Y. Pan, S. Lee, Z. Song, B. S. Han, and V. B. Saputra, "Appearance-based brake-lights recognition using deep learning and vehicle detection," in *IEEE Int. Veh. Symp. (IV'16)*, Gothenburg, Sweden, June 2016.

[4] B. Wu, A. Wan, X. Yue, and K. Keutzer, "Squeezeseg: Convolutional neural nets with recurrent CRF for real-time road-object segmentation from 3D LiDAR point cloud," in *IEEE Int. Conf. Robot. Autom. (ICRA'2018), Brisbane, Australia*, May 2018.

[5] B. Tian, R. Chen, Y. Yao, and N. Li, "Robust traffic sign detection in complex road environments," in *IEEE International Conference on Vehicular Electronics and Safety (ICVES'16), Stockholm, Sweden*, July 2016.

[6] L. Chen, X. Hu, W. Tian, H. Wang, D. Cao, and F. Wang, "Parallel planning: a new motion planning framework for autonomous driving," *IEEE/CAA J. Auto. Sini.*, vol. 6, no. 1, pp. 236–246, Jan. 2019.

[7] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," *IEEE Comp. Society Conf. Comp. Vision Pattern Recognition (CVPR'05), San Diego, CA, USA*, June 2005.

[8] S. Bougharriou, F. Hamdaoui, and A. Mtibaa, "Linear SVM classifier based HOG car detection," in *Int. Conf. Sci. Tech. Automatic Control Comp. Eng. (STA'17)*, Monastir, Tunisia, Dec. 2017.

[9] H. Bristow and S. Lucey, "Why do linear SVMs trained on HOG features perform so well," *arXiv e-prints, CoRR abs/1406.2419*, June 2014.

[10] Joseph Redmon et al., "You only look once: Unified, real-time object detection," *CoRR*, vol. abs/1506.02640, June 2015.

[11] W. Liu and D. Anguelov, "SSD: Single shot multibox detector," *arXiv e-prints, CoRR abs/1512.02325*, Dec. 2016.